FitBuddy-AI Fitness Plan Generator Using Gemini Models

Government Arts and Science College-Manur-MSU142

TeamLeader:

Sandhiya P NM ID:F24D0CD882B6C7E9B9F44821FF74E70B

Team Members:Santhanamari G NM ID: C1A1D3256DDD23E87C0472B52EDBF20B

Saraswathi P NM ID: F7D8EF3617B74F0922A414AB1EDEF80E

Chinnathai S NM ID:6A593ADCC611910BA8240846346DA660

Muthumari M NM ID: 61F719397D001D3BE07494880A4743B7

Project Design Phase Phase

Here's the *Project Design Phase* for FitBuddy – AI Fitness Plan Generator, structured to show how the system evolves from requirements into a concrete design:

🎨 Project Design Phase: FitBuddy

1. System Architecture Design

- *Frontend (Presentation Layer)*

  - HTML + CSS + Jinja2 templates for user input and plan display.

  - Pages: index.html (form), result.html (plan + nutrition tip + feedback), all_users.html (admin dashboard).

  - Responsive design with gym-themed visuals.

- *Backend (Application Layer)*

  - FastAPI framework for routing and request handling.

  - Core files: main.py, routes.py, database.py.

  - Integration with Gemini APIs for workout/nutrition generation.

- *AI Layer*
  - Gemini 1.5 Pro → structured 7-day workout plans + feedback updates.
  - Gemini Flash → quick nutrition/recovery tips.
  - Modular functions: gemini_generator.py, gemini_flash_generator.py, updated_plan.py.

- *Database Layer*
  - SQLite with SQLAlchemy ORM.
  - Tables: User, WorkoutPlan.
  - Stores original and updated plans separately for comparison.

---

2. Module-Level Design

- *Workout Plan Generator*
  - Function: generate_workout_gemini()
  - Input: user goal + intensity.
  - Output: formatted 7-day plan (warm-up, main workout, cooldown).

- *Nutrition Tip Generator*
  - Function: generate_nutrition_tip_with_flash()
  - Input: fitness goal.
  - Output: concise nutrition/recovery tip.

- *Feedback Updater*

  - Function: update_workout_plan()

  - Input: original plan + user feedback.

  - Output: revised plan with minimal disruption to structure.

- *Database Manager*

  - Functions: save_user(), save_plan(), update_plan(), get_user().

  - Ensures persistence and retrieval of user data and plans.

- *Admin Dashboard*

  - Route: /view-all-users.

  - Template: all_users.html.

  - Displays user info, original plan, updated plan.

---

3. UI/UX Design

- *Homepage (index.html)*

  - Simple form with labeled fields (name, age, weight, goal, intensity).

  - “Generate Plan” button triggers backend logic.

- *Result Page (result.html)*

  - Displays personalized workout plan + nutrition tip.

  - Feedback form for plan updates.

- *Admin Page (all_users.html)*

  - Tabular view of all users.

- Side-by-side comparison of original vs. updated plans.

---

4. Data Flow Design

1. User submits form → FastAPI captures input.

2. Backend calls Gemini Pro (workout) + Gemini Flash (nutrition).

3. Plan + tip stored in SQLite DB.

4. Result displayed via Jinja2 template.

5. Feedback submitted → Gemini Pro revises plan → DB updated → refreshed UI.

---

5. Security & Deployment Design

- *Security*

  - API keys stored in environment variables.

  - Input validation via Pydantic schemas.

  - Error handling for failed AI calls.

- *Deployment*

  - Local deployment with FastAPI + Uvicorn.

  - Future-ready for cloud deployment (Docker/Kubernetes).

  - API docs available at /docs.